Operaciones CRUD con Hibernate

Inserción

- Para insertar una nueva tupla se utiliza el método persist()
- Hibernate programa la correspondiente operación INSERT, que se ejecutará al sincronizar la sesión con la BD, normalmente al realizar el commit()
- Si los datos no cumplen alguna restricción definida en la BD (CP, UNIQUE, CHECK, etc.), se producirá una excepción

```
Cliente nuevoCliente = new Cliente("CL999", "Paco", "Tilla", "600999999",
                                   LocalDate.of(1990, 4, 15), "paco.tilla@correo.es", "ES", "GENERAL");

sesion.persist(nuevoCliente);
tr.commit();
```

LocalDate.of(año, mes, día) crea un objeto LocalDate a partir de una fecha concreta.
LocalDate.of(1990, 4, 15) representa el 15/04/1990

Borrado

- Para eliminar una tupla se utiliza el método remove()

```
Cliente cliente = sesion.find(Cliente.class, "CL999");
if (cliente != null)
   sesion.remove(cliente);
tr.commit();
```

Operaciones CRUD con Hibernate

Actualización

- Para actualizar una entidad gestionada por la sesión, se modifican sus atributos mediante los métodos set(). Hibernate detecta automáticamente los cambios y los refleja en la BD al sincronizar la sesión, normalmente al realizar commit()

```
Cliente clienteActualizado = sesion.find(Cliente.class, "CL001");
clienteActualizado.setEmail("nuevoCorreo@correo.es");
tr.commit();
```

- Para actualizar una tupla que no está en el entorno de una sesión (por ejemplo, porque viene de otra sesión, capa de aplicación, etc.), se debe utilizar el método merge(). La actualización será efectiva en la BD tras un commit()

```
Cliente cliente = objetoDeCapaAplicacion();   // Cliente previamente obtenido de otra sesión
Cliente clienteActualizado = sesion.merge(cliente);

clienteActualizado.setEmail("nuevoCorreo@correo.es");
tr.commit();
```

merge() copia el estado del objeto recibido en una entidad gestionada por la sesión y devuelve esa entidad. El objeto original permanece separado (**detached**). Por tanto, los cambios posteriores deben realizarse sobre el objeto devuelto por merge()

Mapeo de relaciones

Relaciones "uno a muchos"

- En *Hibernate*, una relación **uno a muchos** puede representarse de forma **bidireccional**, permitiendo navegar por la relación en ambos sentidos

- En el lado **1**, un objeto contiene una colección de objetos del lado **n** mediante **@OneToMany**
- En el lado **n**, cada objeto contiene una referencia al objeto del lado **1** mediante **@ManyToOne**. Este lado es el propietario de la relación porque contiene la clave ajena

Guia.java

mappedBy = "guia" indica que la relación se gestiona desde el atributo **guia** de **Turno.java**, donde se encuentra mapeada la clave ajena **idGuia**

```
@OneToMany(mappedBy = "guia")
private Set<Turno> turnos = new HashSet<Turno>();
```

Turno.java

@ManyToOne indica que cada turno tiene un único guía
@JoinColumn especifica que la relación se almacena mediante la clave ajena **idGuia** de la tabla GUIA

```
@JoinColumn(name = "idGuia", referencedColumnName = "idGuia")
@ManyToOne(optional = false)
private Guia guia;
```

Mapeo de relaciones

- En el modelo conceptual inicial, la entidad TURNO es una entidad débil dependiente de AVENTURA. Al introducir la clave sustituta idTurno, la entidad pasa a tener entidad propia, pero se mantiene la relación uno a muchos entre AVENTURA y TURNO

- En el modelo relacional, la relación se ha transformado propagando la clave **idAventura** hacia la tabla TURNO

Aventura.java

```
@OneToMany(mappedBy = "aventura")
private Set<Turno> turnos = new HashSet<Turno>();
```

mappedBy = "aventura" indica que la relación se gestiona desde el atributo aventura de **Turno.java**

Turno.java

```
@JoinColumn(name = "idAventura", referencedColumnName = "idAventura")
@ManyToOne(optional = false)
private Aventura aventura;
```

@ManyToOne indica que cada turno pertenece a una única aventura
@JoinColumn especifica que la relación se almacena mediante la clave ajena **idAventura** de la tabla TURNO

Mapeo de relaciones

Creación de un nuevo turno asignándole una aventura y un guía existentes en la base de datos

Suponemos que los datos del nuevo turno cumplen las reglas de negocio establecidas

```
try {
   Aventura aventura = sesion.find(Aventura.class, "AV001");
   Guia guia = sesion.find(Guia.class, "G005");

   if (aventura == null || guia == null) {
      System.out.println("La aventura o el guía no existen en la BD");
   } else {
      Turno turno = new Turno("T0061", LocalDateTime.of(2027, 6, 20, 9, 0), (short) 14 );
      turno.setAventura(aventura);
      turno.setGuia(guia);
      aventura.getTurnos().add(turno);
      guia.getTurnos().add(turno);
      sesion.persist(turno);
      tr.commit();
   }

} // completar con catch y finally
```

Mediante **setAventura()** y **setGuia()** de la clase **Turno** establecemos las relaciones del nuevo turno con la aventura y el guía. **Turno** es el lado propietario de ambas relaciones, así que Hibernate usa estos objetos para establecer las claves ajenas **idAventura** e **idGuia**

Se añade el nuevo turno a las colecciones de **Aventura** y **Guia** para mantener sincronizados en memoria ambos lados de las relaciones bidireccionales

persist() hace persistente el nuevo turno y realiza el almacenamiento en la base de datos

Mapeo de relaciones

Actualización del guía asignado a un turno

```
try {
   // Recuperamos el nuevo guía
   Guia nuevoGuia = sesion.find(Guia.class, "G008");
   if (nuevoGuia == null) {  System.out.println("El guía no existe en la BD");
   } else {
      // Recuperamos el turno que queremos modificar
      Turno turno = sesion.find(Turno.class, "T0061");
      if (turno == null) {   System.out.println("El turno no existe en la BD");
      } else {
         // Guardamos una referencia al guía actual
         Guia guiaAnterior = turno.getGuia();

         // Se modifica el lado propietario de la relación
         turno.setGuia(nuevoGuia);

         // Se mantienen sincronizados ambos lados de la relación
         if (guiaAnterior != null) {   guiaAnterior.getTurnos().remove(turno);   }
         nuevoGuia.getTurnos().add(turno);

         tr.commit();
      }
   }
} // completar con catch y finally
```

Se actualizan también las colecciones de los dos guías para mantener sincronizados en memoria ambos lados de la relación bidireccional

Turno es el lado propietario de la relación, así que **turno.setGuia(nuevoGuia)** es la operación que modifica la clave ajena TURNO.idGuia. Al hacer commit(), Hibernate ejecuta el UPDATE correspondiente, actualizando **idGuia** en la tabla **TURNO**

Mapeo de relaciones

Listado de turnos junto con cierta información de la aventura y el guía asignados

```
try {

   Query<Turno> consulta = sesion.createQuery("SELECT t FROM Turno t", Turno.class );

   List<Turno> turnos = consulta.getResultList();

   for (Turno turno : turnos) {
      System.out.println(
            turno.getFechaHora() + "\t"
            + turno.getAventura().getNombre() + "\t"
            + turno.getGuia().getNombre() + "\t"
            + turno.getGuia().getEmail()
      );
   }

   tr.commit();

} // completar con catch y finally
```

A partir de cada objeto **Turno** podemos navegar por las relaciones mapeadas. **getAventura()** devuelve el objeto **Aventura** asociado y **getGuia()** devuelve el objeto **Guia**. A partir de ellos podemos acceder a sus atributos mediante sus métodos **get()**

Mapeo de relaciones

Listado de los guías y los turnos que tienen asignados

```
try {
   Query<Guia> consulta = sesion.createQuery("SELECT g FROM Guia g", Guia.class );
   List<Guia> guias = consulta.getResultList();

   for (Guia guia : guias) {
      System.out.println(guia.getNombre());
      Set<Turno> turnos = guia.getTurnos();

      for (Turno turno : turnos) {
         System.out.println("\t" + turno.getIdTurno() + "\t"
               + turno.getFechaHora() + "\t" + turno.getAventura().getNombre()  );
      }
   }

   tr.commit();

} // completar con catch y finally
```

Desde un objeto **Guia** podemos acceder mediante **getTurnos()** al conjunto de objetos **Turno** asociados. Cada elemento de la colección es un objeto completo y permite, a su vez, navegar hacia otras entidades relacionadas como, por ejemplo, **turno.getAventura().getNombre()**

Mapeo de relaciones

Relaciones "muchos a muchos" con atributos

- En el modelo conceptual, un cliente puede realizar muchas reservas y un turno puede ser reservado por muchos clientes. Además, la relación tiene información propia: fechaReserva, importe, estado y valoración
- En estos casos, Hibernate mapea la tabla intermedia RESERVA como una entidad Java independiente

- La relación "muchos a muchos" se representa en Hibernate mediante dos relaciones "uno a muchos":
  - CLIENTE uno a muchos hacia RESERVA
  - TURNO uno a muchos hacia RESERVA

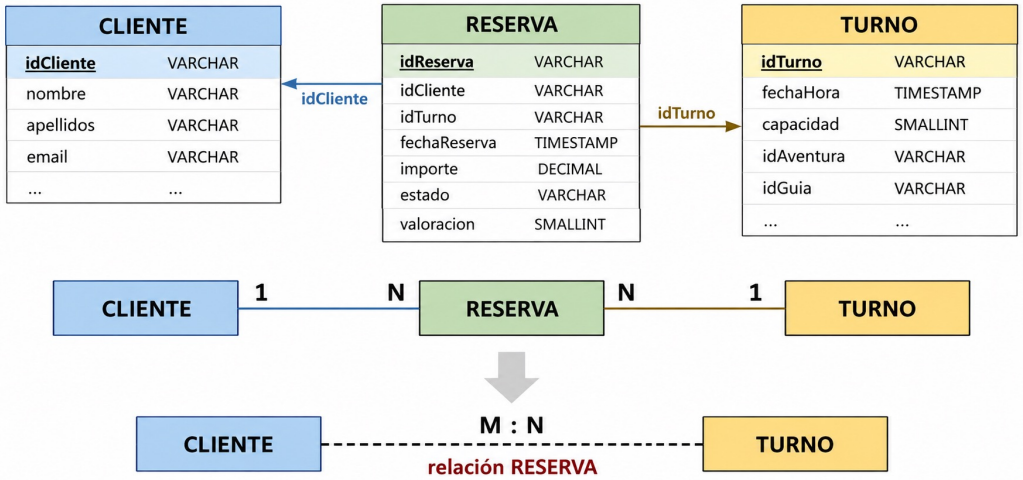


Imagen que representa la relación RESERVA a nivel de base de datos

Mapeo de relaciones

Mapeo desde Reserva.java

```
@JoinColumn(name = "idCliente", referencedColumnName = "idCliente")
@ManyToOne(optional = false)
private Cliente cliente;

@JoinColumn(name = "idTurno", referencedColumnName = "idTurno")
@ManyToOne(optional = false)
private Turno turno;
```

Cada objeto Reserva está asociado a un único Cliente
@JoinColumn utiliza idCliente como Clave Ajena hacia CLIENTE

Cada objeto Reserva está asociado a un único Turno
@JoinColumn utiliza idTurno como Clave Ajena hacia TURNO

En la tabla RESERVA, idCliente e idTurno son Claves Ajenas que contienen identificadores. En la clase Reserva, Hibernate representa esas relaciones mediante objetos Cliente y Turno

@JoinColumn establece la correspondencia entre el atributo objeto y la Clave Ajena de la tabla

Mapeo de relaciones

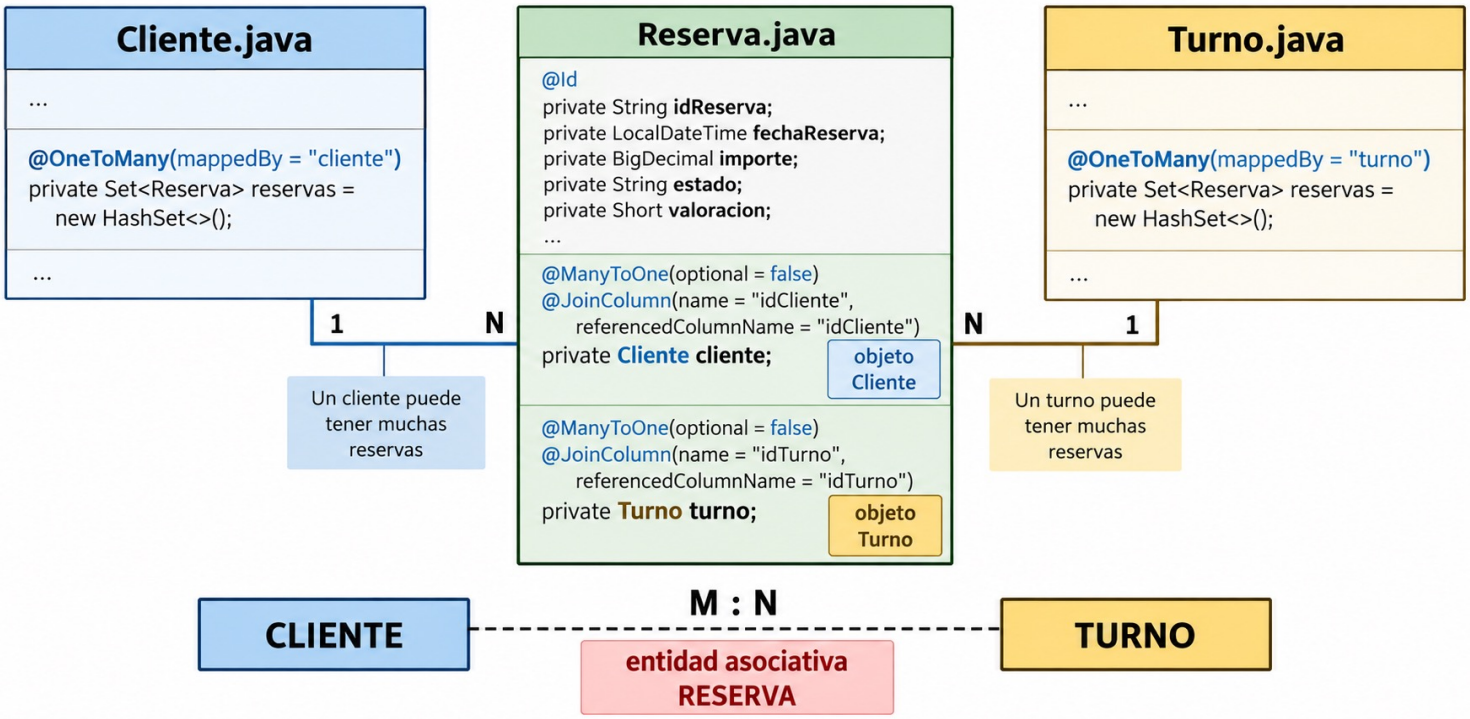
Mapeo desde Cliente.java

```
@OneToMany(mappedBy = "cliente")
private Set<Reserva> reservas = new HashSet<>();
```

Mapeo desde Turno.java

```
@OneToMany(mappedBy = "turno")
private Set<Reserva> reservas = new HashSet<>();
```

mappedBy="cliente" y mappedBy="turno" indican la asociación con loa atributos **cliente** y **turno** de Reserva.java



Mapeo de relaciones

Listado de clientes y de los turnos que tienen reservados

```
try {
   Query consulta = sesion.createNativeQuery("SELECT * FROM CLIENTE", Cliente.class);
   List<Cliente> clientes = consulta.getResultList();

   for (Cliente cliente : clientes) {
      System.out.println(cliente.getNombre());
      Set<Reserva> reservas = cliente.getReservas();

      for (Reserva reserva : reservas) {
         Turno turno = reserva.getTurno();
         System.out.println( turno.getIdTurno() + "\t" + turno.getFechaHora() );
      }
   }

   tr.commit();

} // completar con catch y finally
```

Mapeo de relaciones

Listado de turnos y de los clientes que los tienen reservados

```
try {
   Query consulta = sesion.createNativeQuery("SELECT * FROM TURNO", Turno.class);
   List<Turno> turnos = consulta.getResultList();

   for (Turno turno : turnos) {
      System.out.println(turno.getIdTurno() + "\t" + turno.getFechaHora() );
      Set<Reserva> reservas = turno.getReservas();

      for (Reserva reserva : reservas) {
         Cliente cliente = reserva.getCliente();
         System.out.println(cliente.getNombre() + "\t" + cliente.getApellidos() );
      }
   }

   tr.commit();

} // completar con catch y finally
```

Mapeo de relaciones

Creación de una nueva reserva

- Para insertar una Reserva es suficiente establecer sus relaciones con Cliente y Turno desde el lado propietario (@ManytoOne) y hacer persistente el objeto Reserva

```
try {
   Cliente cliente = sesion.find(Cliente.class, "CL001");
   Turno turno = sesion.find(Turno.class, "T0001");
   if (cliente != null && turno != null) {

      Reserva reserva = new Reserva("R0016", LocalDateTime.now(),
                new BigDecimal("28.00"), "PENDIENTE" );

      reserva.setCliente(cliente);
      reserva.setTurno(turno);

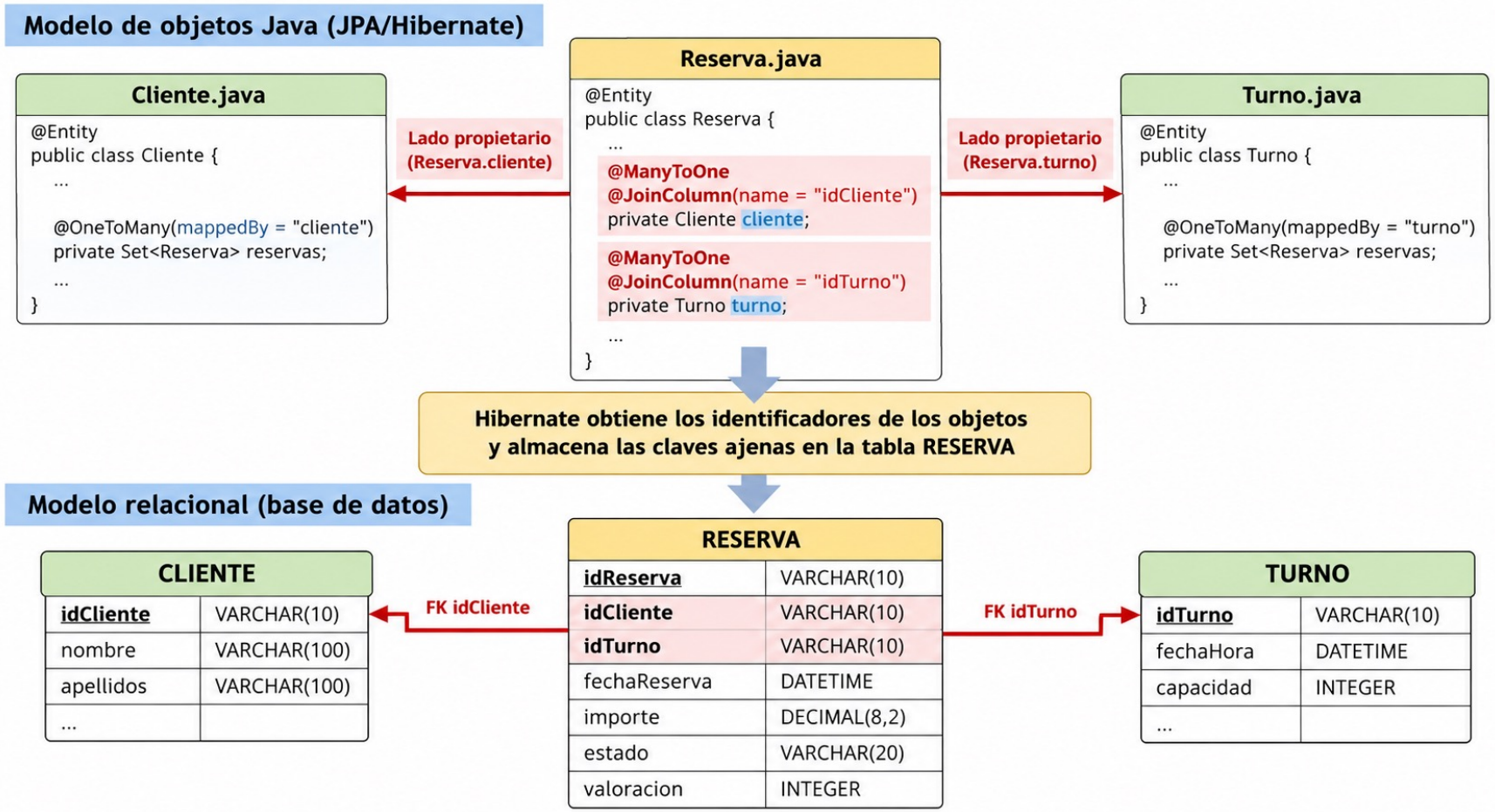
      sesion.persist(reserva);
      tr.commit();
   }

} // completar con catch y finally
```

Reserva.java es el lado propietario de ambas relaciones. Al asignar los objetos Cliente y Turno, Hibernate obtiene sus identificadores y almacena las claves ajenas idCliente e idTurno

Mapeo de relaciones

Correspondencia entre el modelo de objetos Java y el modelo relacional



Mapeo de relaciones

SOLO PERSISTENCIA

Se asignan las relaciones desde el lado propietario y se hace persistente la nueva reserva.

```
reserva.setCliente(cliente);
reserva.setTurno(turno);
sesion.persist(reserva);
```

BD correctamente actualizada

CLIENTE	
idCliente	**CL001**
nombre	Ana
apellidos	García
...	

FK idCliente

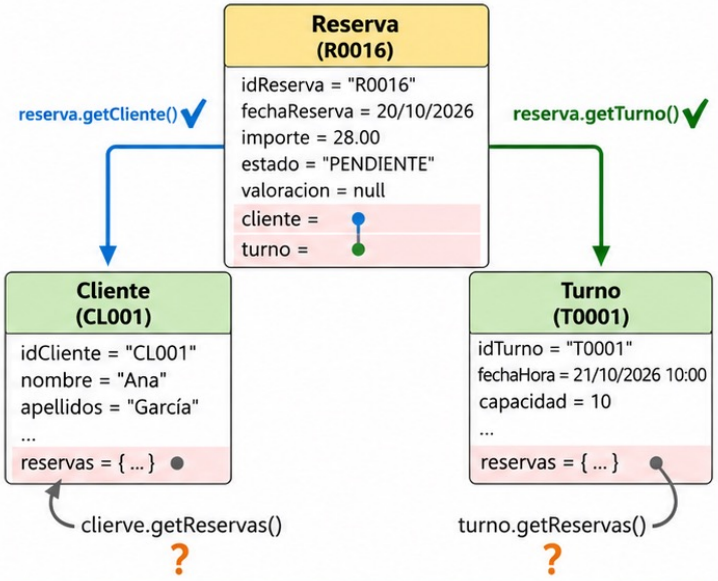
RESERVA	
idReserva	R0016
idCliente	CL001
idTurno	T0001
fechaReserva	20/10/2026
importe	28.00
estado	PENDIENTE
valoracion	null

FK idTurno

TURNO	
idTurno	T0001
fechaHora	21/10/2026 10:00
capacidad	10
...	

OBJETOS EN MEMORIA

Estado de los objetos Java después de ejecutar el código anterior en la misma sesión.



- Las referencias desde Reserva hacia Cliente y Turno están establecidas (correctas).
- Las colecciones cliente.getReservas() y turno.getReservas() no se han actualizado explícitamente. Pueden no contener la nueva reserva en este momento.
- La base de datos sí refleja correctamente la relación.

Mapeo de relaciones

Creación de una reserva manteniendo sincronizados ambos lados de las relaciones

```
try {
   Cliente cliente = sesion.find(Cliente.class, "CL001");
   Turno turno = sesion.find(Turno.class, "T0001");
   if (cliente != null && turno != null) {

      Reserva reserva = new Reserva("R0016", LocalDateTime.now(), new BigDecimal("28.00"),
                                                    "PENDIENTE" );


      // Lados propietarios
      reserva.setCliente(cliente);
      reserva.setTurno(turno);

      // Lados inversos
      cliente.getReservas().add(reserva);
      turno.getReservas().add(reserva);

      sesion.persist(reserva);
      tr.commit();
   }

} // completar con catch y finally
```

Con estos métodos se mantienen actualizados y sincronizados los lados inversos, de forma que los objetos Cliente y Turno que ya están en memoria también contienen la nueva reserva

Mapeo de relaciones

PERSISTENCIA + SINCRONIZACIÓN

Se establecen las relaciones desde el lado propietario, se actualizan los lados inversos y se hace persistente la nueva reserva.

```
reserva.setCliente(cliente);
reserva.setTurno(turno);
cliente.getReservas().add(reserva);
turno.getReservas().add(reserva);
sesion.persist(reserva);
```

BD correctamente actualizada

CLIENTE	
idCliente	**CL001**
nombre	Ana
apellidos	García
...	...

FK idCliente

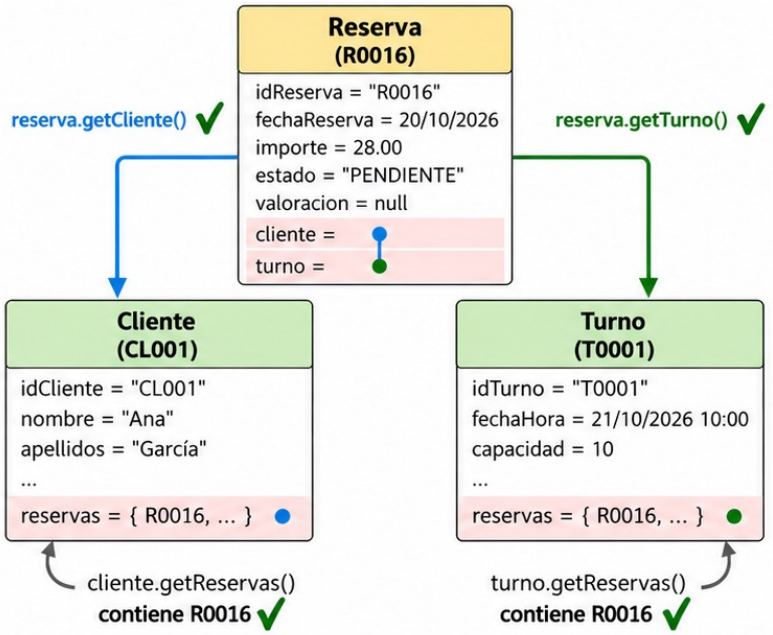
RESERVA	
idReserva	R0016
idCliente	CL001
idTurno	T0001
fechaReserva	20/10/2026
importe	28.00
estado	PENDIENTE
valoracion	null

FK idTurno

TURNO	
idTurno	**T0001**
fechaHora	21/10/2026 10:00
capacidad	10
...	...

OBJETOS EN MEMORIA

Estado de los objetos Java después de ejecutar el código anterior en la misma sesión.



- Las referencias desde Reserva hacia Cliente y Turno están establecidas.
- Las colecciones de Cliente y Turno también contienen la nueva Reserva.
- La base de datos y el grafo de objetos en memoria están sincronizados.

Mapeo de relaciones

Borrado de una reserva manteniendo sincronizados los objetos en memoria

```
try {
   Reserva reserva = sesion.find(Reserva.class, "R0017");
   if (reserva != null) {

      Cliente cliente = reserva.getCliente();
      Turno turno = reserva.getTurno();

      // Se actualizan los objetos en memoria
      cliente.getReservas().remove(reserva);
      turno.getReservas().remove(reserva);

      // Se elimina la entidad de la BD
      sesion.remove(reserva);

      tr.commit();
   }

} // completar con catch y finally
```

remove(reserva) elimina la tupla correspondiente de la base de datos

Si la aplicación va a seguir utilizando los objetos Cliente y Turno dentro de la misma sesión, eliminamos también la reserva de sus colecciones para mantener sincronizado el modelo de objetos